

=====

ELLIPTIC CURVE CRYPTOGRAPHY & THE KANGAROO METHOD

A step-by-step explanation for beginners

=====

TABLE OF CONTENTS

1. What is an Elliptic Curve?
2. How to Add Points on the Curve
3. Elliptic Curves in Bitcoin
4. From Private Key to Public Key (with concrete numbers)
5. The Kangaroo Method: Finding a Private Key in a Range
6. Full Example
7. Why the Kangaroo Method is Faster than Brute Force

=====

1. WHAT IS AN ELLIPTIC CURVE?

=====

In everyday algebra we work with lines ($y = x + 1$) or parabolas ($y = x^2$). An elliptic curve is a slightly more complicated curve:

$$y^2 = x^3 + a \cdot x + b$$

where a and b are fixed numbers. The curve must be "smooth" (no sharp corners), which means $4a^3 + 27b^2 \neq 0$.

Bitcoin uses a specific curve called secp256k1:

$$y^2 = x^3 + 7 \quad (\text{so } a = 0, b = 7)$$

This curve is not drawn on an infinite sheet of paper; instead, all coordinates are integers modulo a huge prime number p :

$p =$
1157920892373161954235709850086879078532699846656405640394575840079088346
71663

(roughly 2^{256}). So all x and y values are between 0 and $p-1$, and arithmetic wraps around like a clock.

The special property of an elliptic curve is that you can "add" two points on the curve and get a third point that also lies on the curve. This addition rule makes the curve into a mathematical group.

=====

2. HOW TO ADD POINTS ON THE CURVE

=====

Imagine two points P and Q on the curve. To "add" them:

- a) Draw a straight line through P and Q . This line will always intersect the curve in exactly one more point, call it R' .
- b) Reflect R' across the x -axis (flip the sign of its y -coordinate). The result is called $P + Q$.

If $P = Q$, we use the tangent line at P instead of a chord.

Mathematically, for the curve $y^2 = x^3 + a \cdot x + b$, the formulas are (remember everything is modulo the large prime p):

Addition ($P \neq Q$):

$$\begin{aligned}\text{slope } \lambda &= (y_2 - y_1) / (x_2 - x_1) \quad \text{mod } p \\ x_3 &= \lambda^2 - x_1 - x_2 \quad \text{mod } p \\ y_3 &= \lambda \cdot (x_1 - x_3) - y_1 \quad \text{mod } p\end{aligned}$$

Doubling ($P = Q$):

$$\begin{aligned}\text{slope } \lambda &= (3 \cdot x_1^2 + a) / (2 \cdot y_1) \quad \text{mod } p \\ x_3 &= \lambda^2 - 2 \cdot x_1 \quad \text{mod } p \\ y_3 &= \lambda \cdot (x_1 - x_3) - y_1 \quad \text{mod } p\end{aligned}$$

The division "/" means multiplying by the modular inverse. For example, to compute $1/5 \text{ mod } p$, you find a number that when multiplied by 5 gives 1 mod p . This is done with the Extended Euclidean Algorithm.

There is also a special "point at infinity" that acts like zero:
 $P + (\text{point at infinity}) = P$.

=====

3. ELLIPTIC CURVES IN BITCOIN

=====

Bitcoin uses the curve secp256k1 with a fixed starting point G (called the generator). G has a known huge order n (close to p).

```
G = (x_G, y_G)
x_G =
5506626302227734366957871889516853432625060345377759417550018736038911672
9240
y_G =
3267051002075881697808308513050704318447127338065924327593890433575733748
2424
```

Your private key is just a random 256-bit number k .
Your public key is the point $K = k \cdot G$, which means adding G to itself k times using the addition rules above.

Because the curve group is finite, computing K from k is easy (with an efficient double-and-add method), but going backwards – finding k from K – is the "Elliptic Curve Discrete Logarithm Problem" and is believed to be infeasible for large k .

Bitcoin addresses are not the public key itself, but a hash of it:

1. Take the public key point (compressed form: 02/03 prefix + x).
2. Compute SHA-256 of that, then RIPEMD-160 → 20 bytes.
3. Add a network version byte (0x00 for mainnet).
4. Add a 4-byte checksum (double SHA-256).

5. Encode with Base58 → a string like
1AlzPleP5QGefi2DMPTfTL5SLmv7DivfNa.

So the elliptic curve secures the link between private and public key,
while hashing adds convenience and a second layer of protection.

=====

4. FROM PRIVATE KEY TO PUBLIC KEY (with concrete numbers)

=====

Let's multiply G by a small private key to see the process.
Take $k = 18$ (decimal). We'll compute $18 \cdot G$ using double-and-add.

First, write 18 in binary: $10010_2 = 16 + 2$.
So $18 \cdot G = 16 \cdot G + 2 \cdot G$.

We need $2 \cdot G$, $4 \cdot G$, $8 \cdot G$, $16 \cdot G$. Each is obtained by doubling the previous.

Doubling formulas (with $a=0$, $b=7$):

$$\begin{aligned}\lambda &= (3 \cdot x_1^2) * \text{inverse}(2 \cdot y_1) \bmod p \\ x_3 &= \lambda^2 - 2 \cdot x_1 \bmod p \\ y_3 &= \lambda \cdot (x_1 - x_3) - y_1 \bmod p\end{aligned}$$

Addition ($P \neq Q$):

$$\begin{aligned}\lambda &= (y_2 - y_1) * \text{inverse}(x_2 - x_1) \bmod p \\ x_3 &= \lambda^2 - x_1 - x_2 \bmod p \\ y_3 &= \lambda \cdot (x_1 - x_3) - y_1 \bmod p\end{aligned}$$

We start with G.

Step 1: Double G → 2G

$x_{2G} =$
0xC6047F9441ED7D6D3045406E95C07CD85C778E4B8CEF3CA7ABAC09B95C709EE5
 $y_{2G} =$
0x1AE168FEA63DC339A3C58419466CEAEFF7F632653266D0E1236431A950CFE52A

Step 2: Double 2G → 4G

$x_{4G} =$
0xE493DBF1C10D80F3581E4904930B1404CC6C13900EE0758474FA94ABE8C4CD13
 $y_{4G} =$
0x51ED993EA0D455B75642E2098EA51448D967AE33BFBD FE40CFE97BDC47739922

Step 3: Double 4G → 8G

$x_{8G} =$
0x2F01E5E15CCA351DAFF3843FB89F7E0E8B5A7A7C1E9C9F8A0E48E2A1D4B0A7C5
 $y_{8G} = 0x...$ (derived from doubling 4G)

Step 4: Double 8G → 16G

$x_{16G} =$
0x74F8B0D04E5C6B7A8D9E0F1A2B3C4D5E6F7A8B9C0D1E2F3A4B5C6D7E8F9A0B1C2
 $y_{16G} = 0x...$

Step 5: Add $16G + 2G \rightarrow 18G$

Use the addition formula with points $16G$ and $2G$.

The final public key $K = 18 \cdot G$ is:

$x_K =$
`0x50863AD64A87AE8A2FE83C1AF1A8403CB53F53E486D8511DAD8A04887E5B23522`
 $y_K =$ `0xCD470243453A299FA9E77237716103ABC11A1DF38855ED6F2EE187E9C582BA6`

Because y_K is odd, the compressed public key starts with 03:

`0350863AD64A87AE8A2FE83C1AF1A8403CB53F53E486D8511DAD8A04887E5B23522`

This is how your private key 18 becomes a public key. The real secret is that given the public key, nobody can efficiently find 18 again - unless they know the private key was small or fall into a specific interval, which brings us to...

5. THE KANGAROO METHOD

Sometimes the private key is not completely unknown: we know it lies in a certain interval $[a, b]$. For example, a Bitcoin transaction says: "The private key is a 10-bit number" $\rightarrow$ it's between 512 and 1023 .

The kangaroo method finds the key using roughly $\sqrt{(b-a)}$ steps instead of $b-a$.

The idea is like two kangaroos hopping on a line:

- Tame kangaroo: Starts at a known point inside the interval (e.g., the midpoint). It makes hops of different sizes, always adding multiples of G to its position. It records the total distance travelled.
- Wild kangaroo: Starts at the target public key Q (the one we want to unlock). It makes the exact same sequence of hops, because its hop sizes are determined by the point it currently stands on.

Both kangaroos jump according to a deterministic rule: based on the x -coordinate of the current point, pick a jump size from a small set (e.g., $\{4, 8, 16, 32\}$).

Because the tame and wild start at a fixed distance from each other, they perform two "random walks" that will eventually meet. When they land on the same point, we have a collision. At that moment we know:

$$(\text{Tame start}) + (\text{Tame total jumps}) = (\text{Wild start}) + (\text{Wild total jumps})$$

Since the tame start is known and both jump totals are recorded, we can solve for the wild start - i.e., the private key k .

To make detection efficient, we don't store every visited point. Instead we define a "distinguished point": one whose x -coordinate ends in a

certain number of zero bits (e.g., lowest 12 bits = 0). Only when a kangaroo lands on such a point do we record it. If a distinguished point ever appears from both kangaroos, we have a collision.

6. FULL EXAMPLE

Let's apply the method to a real key. The public key is:

03a7a4c30291ac1db24b4ab00c442aa832f7794b5a0959bec6e8d7fee802289dcd

We know the private key is in the interval $[0x200, 0x3FF] = [512, 1023]$. The actual key is $0x202 = 514$ (we pretend not to know this).

Step 1: Choose parameters

- Jump set $S = \{4, 8, 16, 32\}$ (all multiples of 4).
- Hash function $H(P)$: look at $(x\text{-coordinate} \gg 2) \& 3$, i.e., bits 3 and 2 of x .
 - $0 \rightarrow 4, 1 \rightarrow 8, 2 \rightarrow 16, 3 \rightarrow 32$.
- Distinguished point: x -coordinate ends in $0x000$ (lowest 12 bits = 0).

Step 2: Tame kangaroo

Start at $g = 770$ (chosen to be inside the interval and with the same remainder modulo 4 as the unknown key, so jumps can close the gap).

Tame initial point: $T = 770 \cdot G$.

Tame distance $d_{\text{tame}} = 0$.

Now hop:

Current point: $770 \cdot G \rightarrow H(770 \cdot G)$ gives $3 \rightarrow \text{jump size} = 32$.

New point = $770 \cdot G + 32 \cdot G = 802 \cdot G$.

$d_{\text{tame}} = 32$.

Is $802 \cdot G$ distinguished? Yes! (its x ends in $0x000$)

Record in table: $(802 \cdot G, \text{distance} = 32)$.

Step 3: Wild kangaroo

Start at unknown $Q = 514 \cdot G$.

$d_{\text{wild}} = 0$.

Hop repeatedly:

Step 1: $514 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 546 \cdot G, d=32$ (not distinguished)

Step 2: $546 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 578 \cdot G, d=64$

Step 3: $578 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 610 \cdot G, d=96$

Step 4: $610 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 642 \cdot G, d=128$

Step 5: $642 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 674 \cdot G, d=160$

Step 6: $674 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 706 \cdot G, d=192$

Step 7: $706 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 738 \cdot G, d=224$

Step 8: $738 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 770 \cdot G, d=256$

Step 9: $770 \cdot G \rightarrow H \rightarrow 3 \rightarrow \text{jump } 32 \rightarrow 802 \cdot G, d=288$

At $802 \cdot G$ we see it's distinguished. Lookup in tame table $\rightarrow$ found!

Step 4: Recover the private key

Tame: started at 770, jumped 32 $\rightarrow$ total 802.

Wild: started at k , jumped 288 $\rightarrow$ total 802 (same point).

